

Test d'admission au double diplôme de licence
FSEGS

NOM ET PRENOM :

Durée : 1h30

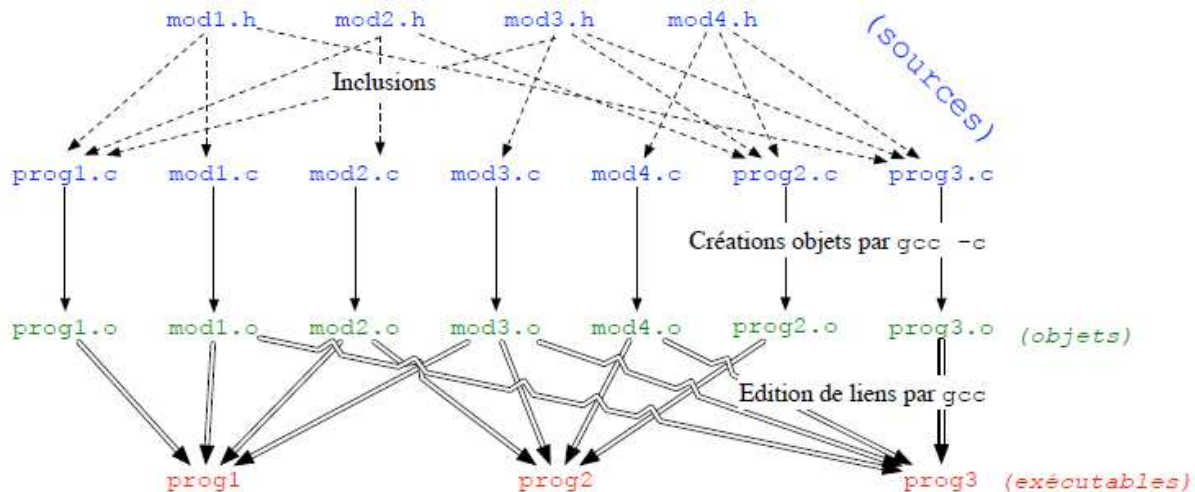
Documents : non autorisés

► PARTIE 1 : Les Commandes Linux (3.25 pts)

Donnez le nom de la commande linux permettant de	
Se déplacer dans une arborescence	
Chercher un document dans une arborescence	
Changer les droits d'accès d'un document	
Décompresser une archive .tar.gz	
Chercher un mot dans un fichier texte	
Lister les processus en cours d'exécution	
Envoyer un signal à un processus	
Indiquez le numéro d'identification de chaque canal	
Canal de sortie standard	
Canal de sortie des erreurs	
Canal d'entrée standard	
Donnez le symbole utilisé pour faire	
Une redirection de la sortie standard vers un fichier nommé result	
Une redirection de la sortie d'erreur vers un fichier nommé errors	
Une redirection de la sortie standard et de la sortie d'erreur vers un fichier nommé both	

► PARTIE 2 : La création d'un Makefile (5pts)

Soit un **projet C** dont le schéma de dépendances entre ses fichiers est le suivant :



Les fichiers « .c » et « .h » sont placés dans le même répertoire comme indiqué par la figure ci-dessous :



Complétez l'écriture du fichier Makefile du projetC permettant de créer les 3 exécutables : prog1, prog2 et prog3.

NB1 : Toute modification d'un fichier « .h » doit engendrer la recompilation des fichiers « .c » l'incluant.

NB2 : Utilisez les variables spéciales \$^,\$< et \$@ en cas de besoin

```
CCLNK = gcc
CCOBJ = gcc -c
prog1 : .....
    $(CCLNK) .....
prog1.o : .....
    $(CCOBJ) .....
prog2 : .....
    $(CCLNK) .....
prog2.o : .....
    $(CCOBJ) .....
```

```

prog3 : .....
      $(CCLNK) .....
prog3.o : .....
      $(CCOBJ) .....
mod1.o : .....
      $(CCOBJ) .....
mod2.o : .....
      $(CCOBJ) .....
mod3.o : .....
      $(CCOBJ) .....
mod4.o : .....
      $(CCOBJ) .....

```

► PARTIE 3 : Programmation système UNIX en C (11.75 pts)

Exercice1 (2.25) : Combien de lignes « Test_DD » affiche chacun des programmes suivants ?

```

#include<stdio.h>
#include<unistd.h>
#include <sys/wait.h>
int main(){
int i ;
for(i=0 ;i<2 ;i++)
    fork();
printf("Test_DD\n");
while(wait(NULL) > 0);
exit(0);}

```

.....4.....

```

#include<stdio.h>
#include<unistd.h>
#include <sys/wait.h>
int main(){
if(fork()==0){
    fork();
    printf("Test_DD\n");}
while(wait(NULL) > 0);
exit(0);}

```

.....2.....

```

#include<stdio.h>
#include<unistd.h>
#include <sys/wait.h>
int main(){
if (fork())
    fork();
printf("Test_DD\n");
while(wait(NULL) > 0);
exit(0);}

```

.....3.....

Exercice 2 (9.5): Complétez le programme **frequence.c** suivant pour qu'il exécute en même temps deux processus :

1) un processus père se déroulant comme suit :

- demande à l'utilisateur de saisir un message (une chaîne de caractères en minuscule)
- envoie le message vers son processus fils via un tube (pipe)

2) un processus fils se déroulant comme suit :

- attend l'arrivée d'un message du père

- calcule puis affiche la fréquence (c.-à-d. le nombre de répétition) de chaque lettre du message. Ces fréquences seront stockées dans un tableau de 26 entiers.

Astuces : si `int freq[26]` est le tableau des fréquences des lettres et si `char c` est une des lettres du message alors on peut incrémenter la fréquence de la lettre `c` par la formule magique `freq[c-'a']++`. On rappelle aussi que le code ascii de 'a' est 97.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

#define LG_MESSAGE 126

int main( int argc , char * argv[] )
{
    int i,freq[26],tab[2];
    char message[LG_MESSAGE] ;

    for(i=0;i<26;i++)
        freq[i]=0;

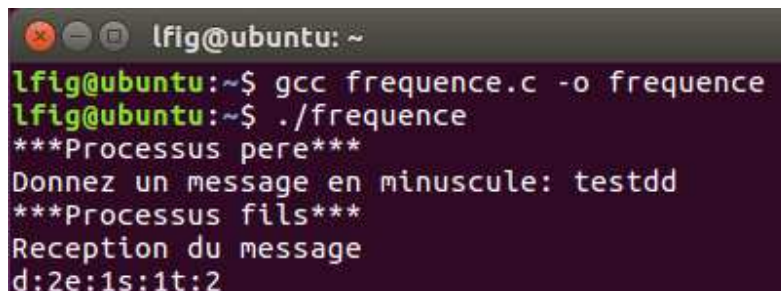
    if(pipe(tab))
    {
        fprintf( stderr, "erreur de creation de tube");
        exit(-1);
    }
    pid_t pid=fork();
    if(pid==-1)
    {
        fprintf( stderr, "Erreur d'execution de fork");
        exit(-1);
    }
    else if (pid==0)//processus fils
    {
        close(tab[1]);.....
        read(tab[0],message,strlen(message)+1);
        close(tab[0]);.....
        for (i=0;message[i]!='\0';i++)
        {
            .....
            freq[message[i]-97]+=1;
            .....
        }
        for (i=0;message[i]!='\0';i++)
        {
            .....
            printf("%c:%d",message[i],freq[message[i]-97]);
            .....
        }
        .....
        .....
        .....
    }
}
```

```

.....
.....
.....
.....
.....
.....
.....
.....
.....
}
else //processus père
{
    close(tab[0]);
    printf("le message doit etre en minuscule"); .....
    scanf("%s",message)
    int verif=1; .....
    while(verif)
    { .....
        for(i=0;message[i]!='0';i++)
        { .....
            if(message[i]<97)
            {scanf("%s",message); .....
                i=0;
            } .....
        }
        verif=0; .....
    }
    write(tab[1],strlen(message)+1); .....
    close(tab[1]);
}
while(wait(NULL) > 0);
exit(0);
}

```

Voici un exemple d'exécution du programme **frequence.c** sous un environnement linux



```

lfig@ubuntu: ~
lfig@ubuntu:~$ gcc frequence.c -o frequence
lfig@ubuntu:~$ ./frequence
***Processus pere***
Donnez un message en minuscule: testdd
***Processus fils***
Reception du message
d:2e:1s:1t:2

```